DAY 02 · STRINGS

Strings

Slicing, methods, formatting, and the text algorithms the exam loves.

2–3 hours / day02_strings / Text processing

INSIDE THIS BOOKLET

Slicing · Methods · f-strings · Scanning

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.



Read & type
Type every example. Predict the output first.



Practice
Implement or debug one test at a time.



Test
Run the suite. Read the first failing line.



Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day02_strings/` · the course runner `python run.py 2`

In this session

Why a whole day on strings? 3

1 What a string is 3

2 The methods you must know 4

3 Characters as numbers: `ord` and `chr` 9

4 Building text with f-strings 10

5 Building a long string piece by piece 11

6 Walking through a string: three patterns 12

7 Special characters inside strings 13

8 The toolbox: small techniques the exercises rely on 14

Exercises 17

End-of-day checklist 18

> **◎ Today's goal**
>
> Handle any string manipulation without having to look anything up. A lot of exam bugs live in string handling: wrong `split`, `strip` misunderstood, off-by-one slices, forgotten case handling.

Why a whole day on strings?

Look at the kinds of programs these exams use: tokenizers that cut text into pieces, parsers that read a command like `SET name Ana`, encoders that turn text into bytes, formatters that produce a report. All of them take text in, pick it apart, and build new text. Even when the core of the algorithm is about numbers or graphs, the input usually arrives as a string that has to be split and converted first. Python has a lot of string methods, and several come in near-identical pairs that behave slightly differently: `split()` and `split(" ")`, `find` and `index`, `strip` and `removeprefix`. Swapping one for its twin is an ideal planted bug: the code still looks right and still works on most inputs. Today you'll learn exactly where the twins differ. As yesterday: keep the REPL open and type every example.

1 What a string is

1.1 A sequence of characters

A **string** (`str`) is a piece of text: a sequence of characters, written between quotes. Single and double quotes mean the same thing: `'hello'` and `"hello"` are identical. Because a string is a *sequence*, everything you learned about list positions and slices on Day 1 works on strings too:

```
>>> s = "hello"
>>> s[0]          # first character
'h'
>>> s[-1]         # last character
'o'
>>> s[1:3]        # characters at positions 1 and 2 (stops before 3)
'el'
>>> s[::-1]       # reversed
'olleh'
>>> len(s)        # number of characters
5
```

Each character is really a number behind the scenes, called its **code point**. `'h'` is 104 and `'é'` is 233. Section 3 shows how to see those numbers, and Day 5 shows how they become bytes in a file.

1.2 Strings can't be changed

Unlike a list, a string can never be modified once it exists. Programmers say strings are **immutable**. If you try to change one character, Python refuses:

```
>>> s = "hello"
>>> s[0] = "H"
TypeError: 'str' object does not support item assignment
```

To get "Hello", you build a **new** string from pieces and point the name `s` at it:

```
>>> s = "H" + s[1:]
>>> s
'Hello'
```

(+ joins two strings together. That's called *concatenation*.)

1.3 So string methods return a new string

Immutability has a consequence that causes real bugs. Every string method that "changes" a string actually leaves the original alone and **returns a new string**. If you don't keep the returned value, the work is lost.

Strings are immutable

Methods never modify the string, they return a new one:

```
s.upper()       # returns 'HELLO'; s is still 'hello'
s = s.upper()   # this is what you meant
```

See it for yourself:

```
>>> name = "  ana  "
>>> name.strip()        # strip() removes spaces at both ends...
'ana'
>>> name                # ...but name itself is unchanged
'  ana  '
>>> name = name.strip() # keep the result
>>> name
'ana'
```

No-op method calls

A line like `name.strip()` on its own does nothing. This is a classic planted bug.

Notice that this is the *opposite* of lists. `items.sort()` changes the list and returns `None`, so writing `items = items.sort()` destroys your list. With strings it's the other way round: `name.strip()` returns the result, so you **must** write `name = name.strip()`. When you read exam code, a string method call standing alone on its own line, with nothing receiving its result, is a red flag.

2 The methods you must know

We'll go through the important methods in groups, according to what you'd use them for. For each one, look at the example *and* the explanation, because the surprises are in the details.

2.1 Cutting a string into pieces: `split`

`split` breaks a string into a list of pieces. This is how you turn a line like `"SET name Ana"` into `["SET", "name", "Ana"]`. It has two quite different modes, and this is the single most important detail in this section. **`split()` with no argument** splits on *any amount* of whitespace (spaces, tabs, newlines) and throws away empty pieces. It's "give me the words":

```
>>> "  a  b ".split()
['a', 'b']
>>> "one\ttwo\nthree".split()
['one', 'two', 'three']
```

`split(" ")` with an argument splits on *exactly* that separator, *every single time* it appears. Two spaces in a row produce an empty string between them, and leading or trailing spaces produce empty strings at the ends:

```
>>> "  a  b ".split(" ")
['', '', 'a', '', 'b', '']
```

So which one is right depends on the spec. For human-written text, where people might type two spaces, you want `split()`. For strict formats where an empty field means something, you want the explicit separator. In comma-separated data, `"a,,b"` has three fields and the middle one is empty:

```
>>> "a,,b".split(",")
['a', '', 'b']
```

You can also limit the number of splits with a second argument. This is how you separate a key from a value that might itself contain the separator:

```
>>> "key=value=with=equals".split("=", 1)    # split at most once
['key', 'value=with=equals']
>>> "a,b,c".rsplit(",", 1)                   # rsplit counts from the right
['a,b', 'c']
```

`rsplit` is handy for "the last part", like the extension of a file name: `"photo.final.jpg".rsplit(".", 1)` gives `['photo.final', 'jpg']`.

2.2 Splitting once, safely: `partition`

`partition(sep)` splits at the *first* occurrence of `sep` and always gives back exactly three things: the part before, the separator itself, and the part after.

```
>>> "k:v:w".partition(":")
('k', ':', 'v:w')
>>> "no colon here".partition(":")
('no colon here', '', '')
```

The second example shows why it's useful. When the separator is missing, `partition` doesn't fail, it just returns empty strings, so you can check `if sep:` afterwards.

2.3 Gluing pieces together: `join`

`join` is the reverse of `split`. You call it **on the separator** and give it the list of pieces:

```
>>> "-".join(["a", "b", "c"])
'a-b-c'
>>> ", ".join(["ana", "ben"])
'ana, ben'
>>> "".join(["x", "y", "z"])     # empty separator: just concatenate
'xyz'
```

The separator goes only *between* items, so there's never a stray one at the end. Every item must already be a string. If you have numbers, convert them first:

```
>>> ",".join([1, 2, 3])
TypeError: sequence item 0: expected str instance, int found
>>> ",".join(str(n) for n in [1, 2, 3])
'1,2,3'
```

2.4 Trimming the ends: `strip`, `lstrip`, `rstrip`, `removeprefix`

`strip()` removes whitespace from both ends of a string. `lstrip()` does only the left end, `rstrip()` only the right. These are very common when reading lines from a file, which end with a newline character:

```
>>> "  hi \n".strip()
'hi'
>>> "line of text\n".rstrip()
'line of text'
```

Now the big trap. You can give `strip` an argument, and most people assume it removes that *word* from the ends. It doesn't. **The argument is a set of characters, and `strip` keeps removing any of those characters from the ends, in any order, for as long as it can:**

```
>>> "xyhixy".strip("xy")      # removes x's and y's from both ends
'hi'
>>> "abba_cab".lstrip("ab")   # removes every a and b from the left, until it hits "_"
'_cab'
```

In the second example, if you were expecting "remove the prefix `ab` once", you'd expect `'ba_cab'`. Instead it stripped `a`, `b`, `b`, `a` and stopped at `_`. If you really want to remove an exact prefix or suffix once, use `removeprefix` and `removesuffix`:

```
>>> "abba_cab".removeprefix("ab")
'ba_cab'
>>> "report.txt".removesuffix(".txt")
'report'
```

If the prefix isn't there, they return the string unchanged. Mixing up `lstrip(prefix)` and `removeprefix(prefix)` is one of the most common string bugs you'll be shown.

2.5 Checking the start and end: `startswith`, `endswith`

```
>>> "photo.jpg".endswith(".jpg")
True
>>> "photo.jpg".endswith((".jpg", ".png"))    # a tuple means "any of these"
True
```

2.6 Searching: `in`, `find`, `index`, `count`

The simplest question, "does this text contain that text?", is answered by `in`:

```
>>> "ell" in "hello"
True
```

To find *where* something is, there are two methods that differ only in what happens when it isn't found:

```
>>> "hello".find("l")      # position of the first match
2
>>> "hello".find("z")      # not found: returns -1
-1
>>> "hello".index("z")     # not found: raises an error
ValueError: substring not found
```

`find` returning `-1` is a trap of its own. Remember that `-1` is a valid position in Python, the last character. So if code does `s[s.find(x)]` without checking, and `x` isn't there, it silently reads the last character instead of failing. Correct code always checks `if pos != -1:` first. `count` counts how many times a piece of text appears, **without overlapping**:

```
>>> "aaaa".count("aa")
2
```

You might expect 3 (positions 0, 1 and 2 all start an `"aa"`). But once `count` has matched positions 0–1, it continues searching from position 2, so it finds only two. Counting *overlapping* matches needs your own loop over every start position. That's one of today's exercises.

2.7 Replacing: `replace`

```
>>> "a-b-c".replace("-", "+")
'a+b+c'
>>> "aaa".replace("a", "b", 2)     # third argument: at most 2 replacements
'bba'
```

2.8 Asking questions about characters: `isdigit`, `isalpha`, ...

These methods answer "is every character in this string of a certain kind?" and return `True` or `False`:

```
>>> "123".isdigit()        # all digits?
True
>>> "abc".isalpha()        # all letters?
True
>>> "ab1".isalnum()        # all letters or digits?
True
>>> " \t\n".isspace()      # all whitespace?
True
>>> "".isalpha()           # an EMPTY string answers False to all of them
False
```

They're used constantly in tokenizers and parsers to decide what kind of piece starts at the current position: a number, a word, or a space. Note the empty-string case: it matters when validating input. `isupper()` and `islower()` look only at letters and ignore everything else, so `"A1".isupper()` is `True`.

2.9 Changing case

```
>>> "Hello World".lower()
'hello world'
>>> "Hello World".upper()
'HELLO WORLD'
>>> "hELLO wORLD".title()        # capitalise each word
'Hello World'
>>> "hello world".capitalize()   # capitalise only the first character
'Hello world'
>>> "Hello".swapcase()
'hELLO'
```

The main use of `lower()` is **case-insensitive comparison**. To check whether two words are "the same, ignoring case", compare `a.lower() == b.lower()`. A frequent bug is lowering only one side.

title() gotcha

`"apostrophe's".title()` → `"Apostrophe'S"` — title() treats any non-letter as a word boundary.

That's why the title-case exercise asks you to write your own version instead of trusting `title()`.

2.10 Padding to a fixed width

These are for producing aligned output, or fixed-width strings like 8-digit binary numbers:

```
>>> "7".zfill(3)          # pad with zeros on the left
'007'
>>> "ab".ljust(5, ".")    # pad on the right to width 5
'ab...'
>>> "ab".rjust(5)         # pad on the left (with spaces by default)
'   ab'
>>> "ab".center(6, "*")
'**ab**'
```

2.11 Splitting into lines: `splitlines`

```
>>> "a\nb\r\nc".splitlines()
['a', 'b', 'c']
```

Text files from Windows end their lines with `\r\n` instead of `\n` (`\r` is a "carriage return" character). `splitlines()` handles both, while `split("\n")` would leave a `\r` at the end of each Windows line.

3 Characters as numbers: `ord` and `chr`

Every character has a number, its **code point**. Two built-in functions convert between them: `ord` turns a character into its number, and `chr` turns a number back into a character.

```
>>> ord("a")
97
>>> chr(97)
'a'
>>> ord("A")
65
>>> ord("0")
48
```

The letters `a` to `z` have consecutive numbers (97 to 122), and so do `A` to `Z` (65 to 90) and the digits `0` to `9` (48 to 57). That means you can do arithmetic on letters:

```
>>> chr(ord("a") + 3)      # three letters after 'a'
'd'
>>> ord("e") - ord("a")    # 'e' is letter number 4, counting from 0
4
```

3.1 A worked example: the Caesar cipher

A **Caesar cipher** is a very old way of hiding a message: shift every letter a fixed number of places along the alphabet, wrapping around at the end. With a shift of 3, `a` becomes `d`, `b` becomes `e`, and `z` wraps round to `c`. Here is the line that shifts one lowercase letter `c` by `k` places:

```
chr((ord(c) - ord("a") + k) % 26 + ord("a"))
```

It looks dense, so let's take it apart from the inside out, with `c = "y"` and `k = 3`:

1. `ord(c) - ord("a")` gives the letter's position in the alphabet, counting from 0: `121 - 97 = 24`.
2. `+ k` shifts it: `24 + 3 = 27`.
3. `% 26` wraps around: there are only 26 letters (positions 0–25), so `27 % 26 = 1`.
4. `+ ord("a")` turns the position back into a code point: `1 + 97 = 98`.
5. `chr(98)` is `'b'`.

So `y` shifted by 3 is `b`, as expected (y → z → a → b). Because Python's `%` always gives a non-negative result here (Day 1), a negative `k` also works and decodes the message. Typical bugs in this line are a missing step 1 or step 4, or `% 25` instead of `% 26`.

3.2 Comparing strings

When you compare strings with `<` or `>`, or sort them, Python compares them **character by character, using the code point numbers**. That has two surprising consequences. Uppercase letters (65–90) all have smaller numbers than lowercase letters (97–122), so every uppercase letter comes before every lowercase one:

```
>>> "Z" < "a"
True
>>> sorted(["banana", "Apple", "cherry"])
['Apple', 'banana', 'cherry']
>>> sorted(["banana", "apple", "Cherry"])
['Cherry', 'apple', 'banana']
```

And digits inside strings are compared as characters, not as numbers. `"10"` versus `"9"`: the first characters are `'1'` and `'9'`, and `'1'` is smaller, so `"10"` comes first:

```
>>> "10" < "9"
True
>>> sorted(["10", "9", "100"])
['10', '100', '9']
>>> sorted(["10", "9", "100"], key=int)    # sort by numeric value instead
['9', '10', '100']
```

If a spec says "sorted numerically" and the code sorts strings without `key=int`, that's a bug.

4 Building text with f-strings

4.1 What an f-string is

An **f-string** is a string with the letter `f` in front of the opening quote. Inside it, anything you put in curly braces `{}` is evaluated as Python and its value is inserted into the text:

```
>>> name, n = "Ana", 42
>>> f"{name} has {n} items"
'Ana has 42 items'
>>> f"next year she'll have {n + 1}"
"next year she'll have 43"
```

4.2 Format specifications

After the expression, you can add a colon and a **format specification** that controls how the value is displayed: how many decimals, how wide, aligned which way, in which number base. Specs often require an exact output format ("two decimal places", "an 8-digit binary number"), so you need to be able to read these. **Decimals:** `.2f` means "a float with exactly 2 digits after the point", rounded:

```
>>> x = 3.14159
>>> f"{x:.2f}"
'3.14'
```

Width and alignment: a number gives the minimum width. Numbers line up on the right by default. `<` aligns left and `^` centres. The `|` characters below are just there so you can see the spaces:

```
>>> f"|{42:5d}|{42:<5d}|{42:^5d}|"
'|   42|42   | 42  |'
```

(d means "a whole number in decimal".) **Zero padding:** a `0` before the width pads with zeros instead of spaces:

```
>>> f"{42:05d}"
'00042'
```

Other number bases: `x` gives hexadecimal (base 16), `X` the same in capitals, `b` binary, and `#` adds the `0x`/`0b` prefix. Combined with zero padding, `08b` gives a full 8-digit byte, which you'll use constantly on Day 5:

```
>>> f"{255:x}  {255:X}  {255:#x}  {5:08b}"
'ff  FF  0xff  00000101'
```

Thousands separators:

```
>>> f"{1234567:,}"
'1,234,567'
```

4.3 Two f-string features made for debugging

`!r` inserts the **repr** of a value instead of its normal text. The repr is how Python would write the value in code: strings get quotes, and invisible characters like newlines and trailing spaces become visible.

```
>>> s = "Ana \n"
>>> print(f"[{s}]")
[Ana
]
>>> print(f"[{s!r}]")
['Ana \n']
```

The second version shows that `s` has a trailing space and a newline. The first one hides both. `=` after an expression prints the expression *and* its value, which is the quickest possible debug print:

```
>>> i, total = 3, 17
>>> print(f"{i=} {total=}")
i=3 total=17
```

5 Building a long string piece by piece

When you build a result from many small pieces, for example in an encoder, the clean way is to collect the pieces in a list and `join` them once at the end:

```
parts = []
for x in items:
    parts.append(str(x))
result = ",".join(parts)        # O(n); repeated `s += x` is O(n^2) in the worst case
```

Why not just `result += piece` in the loop? Since strings can't be changed, every `+=` has to create a brand-new string and copy everything accumulated so far into it. With a thousand pieces, that's a lot of copying. Appending to a list is cheap, and `join` copies each character exactly once. (The "O(n)" notation in the comment is a way of describing how the amount of work grows with the input size. Here it means "proportional to the total length", as opposed to "proportional to the square of it".) For exam-sized inputs, both versions work fine. The reason to know the pattern is that you'll see it in well-written code, and it also gets separators right for free: `join` never adds a trailing comma.

6 Walking through a string: three patterns

Many text algorithms move through a string position by position, deciding at each step what's there and how much to consume. There are three patterns you'll meet again and again, in today's exercises, the tokenizer on Day 9, and the parsers on Day 8.

6.1 Pattern 1: the index-pointer loop

Use this when you need to look ahead, or consume a different number of characters at each step. Here it finds **runs**, i.e. stretches of the same character repeated, like the `"aaa"` in `"aaabcc"`:

```
s = "aaabcc"
i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]:
        j += 1
    # s[i:j] is a run of identical characters
    print(s[i:j], j - i)
    i = j          # forgetting this = infinite loop
```

Output:

```
aaa 3
b 1
cc 2
```

How it works: `i` marks where the current run starts. The inner loop moves `j` forward as long as the character at `j` is the same as the one at `i`. When it stops, `s[i:j]` is the whole run and `j - i` is its length. Then `i = j` jumps straight to the start of the next run. Three details make or break this loop:

- **The order of the checks in `j < len(s) and s[j] == s[i]` matters.** and stops as soon as the first part is false. So when `j` reaches the end, `s[j]` is never evaluated. If you swapped the two parts, `s[j]` would be read with `j == len(s)` and crash with `IndexError`.
- **`i` must always move forward.** If `i = j` is missing, `i` never changes and the loop runs forever.
- **It's a `while`, not a `for`,** because the step size changes: sometimes we jump 3, sometimes 1.

6.2 Pattern 2: building a number from digits

When digits are embedded in a larger string, you often read them one at a time and build the number yourself:

```
num = 0
for c in "1234":
    num = num * 10 + int(c)
```

Watch `num` grow: `0` → `0*10+1 = 1` → `1*10+2 = 12` → `12*10+3 = 123` → `123*10+4 = 1234`. Each new digit shifts the existing number one place to the left (×10) and adds itself on the right. You need this in today's RLE decoder, where `"12a"` means "twelve a's", so the 1 and the 2 must be combined into 12. If you replace 10 with 2, the same loop reads binary.

6.3 Pattern 3: two pointers moving toward each other

Some problems are easiest with one position starting at each end, moving toward the middle. A classic example is reversing a list in place by swapping the two ends, then the next two, and so on:

```
def reverse_in_place(items):
    lo, hi = 0, len(items) - 1
    while lo < hi:
        items[lo], items[hi] = items[hi], items[lo]   # swap the two ends
        lo += 1
        hi -= 1
```

For `[1, 2, 3, 4, 5]`: swap positions 0 and 4, giving `[5, 2, 3, 4, 1]`. Then swap positions 1 and 3, giving `[5, 4, 3, 2, 1]`. Now `lo = hi = 2`, so the loop stops, and the middle item stays where it is. The same shape works for any check that compares a sequence with its mirror image, which is what you'll need for the palindrome exercise (a palindrome is a word that reads the same backwards, like "level"). The usual bug is starting `hi` at `len(items)`, which is one past the last position and crashes.

7 Special characters inside strings

Some characters can't be typed directly into a string, like a newline or a tab. Python lets you write them with a backslash followed by a letter. This is called an **escape sequence**:

You type	You get
`"\n"`	a newline (one character)
`"\t"`	a tab (one character)
`"\\"`	one backslash
`"\""`	a double quote inside a double-quoted string

Even though you typed two characters, each escape is *one* character in the string: `len("\n")` is 1. Sometimes you want backslashes to stay as plain backslashes, for example in Windows paths. A **raw string** has an `r` before the quote and turns escapes off: `r"\n"` is two characters, a backslash and an `n`.

See the invisible

`Print(repr(s)).`

`print` *renders* special characters, so a trailing space or a stray `\r` is invisible. `repr` shows them:

```
>>> s = "done \r\n"
>>> print(s)
done
>>> print(repr(s))
'done \r\n'
```

Whenever a test says two strings are different and they *look* identical in the output, compare their `repr`s. The difference is almost always whitespace.

8 The toolbox: small techniques the exercises rely on

The methods in section 2 are the vocabulary. The exercises also combine them with a few general Python techniques that are worth spelling out, because the reference solutions use them without comment. Try each one in the REPL.

8.1 Repeating a string with `*`

Multiplying a string by a whole number repeats it:

```
>>> "ab" * 3
'ababab'
>>> "-" * 10
'----------'
>>> "x" * 0
''
```

This is the quickest way to build "n spaces" (`" " * n`), "n stars" for masking a password (`"*" * n`), or "this character n times" when decoding run-length encoding (`"a" * 12`). The number must be an `int`. `"a" * "3"` is a `TypeError`, so convert first: `"a" * int("3")`.

8.2 Is this character a lowercase letter? Comparing characters

Since characters are compared by their code point (section 3), and the letters a to z have consecutive code points, "is `c` a lowercase ASCII letter?" can be written as a range check, chained the same way as with numbers (Day 1, section 8):

```
>>> c = "q"
>>> "a" <= c <= "z"
True
>>> "A" <= "Q" <= "Z"
True
>>> "a" <= "é" <= "z"          # é is outside the plain a-z range
False
```

That last line is why this check is often preferred over `c.isalpha()` when a spec says "ASCII letters only": `"é".isalpha()` is `True`.

8.3 Getting the position too: `enumerate` on words

Day 1 introduced `enumerate`, which gives you `(position, item)` pairs. It's especially useful when the **first** item is treated differently from the others, like in title case, where a "minor word" such as `"of"` is still capitalised when it starts the title:

```
for i, w in enumerate(s.split(" ")):
    if i == 0:
        ...      # the first word
    else:
        ...      # every other word
```

Or the reverse question, "is this the first character?", in camelCase-to-snake_case conversion: an uppercase letter gets an underscore in front of it, *except* at position 0.

8.4 Unpacking `partition`, and the _ placeholder

`partition` always returns exactly three values (section 2), so you can unpack them straight into three names. You usually don't need the separator itself, so it goes into `_`, the conventional "I don't care" name:

```
>>> key, _, value = "b=x=y".partition("=")
>>> key, value
('b', 'x=y')
>>> key, _, value = "c".partition("=")      # no "=" at all
>>> key, value
('c', '')
```

Notice how this handles two awkward cases for free: the value can contain `=` (only the first one splits), and a missing `=` gives an empty value instead of an error. `"c".split("=")` would give `['c']`, and unpacking that into two names would crash.

8.5 Building a list of tuples

When a function must return pairs, like `[("a", "1"), ("b", "x=y")]`, you build it with append and a tuple:

```
out.append((key, value))
```

The **double** parentheses matter. The outer ones belong to the `append(...)` call, the inner ones create the tuple. `out.append(key, value)` would be an error, because `append` takes exactly one item, and here we want that one item to be a pair.

8.6 Working character by character: a list of characters, then `join`

Strings can't be changed (section 1), so to *build* a string bit by bit, collect the pieces in a list and `join` them at the end (section 5). The same trick works for reversing a word you're collecting:

```
word = []                   # characters of the current word
word.append("c"); word.append("a"); word.append("t")
"".join(reversed(word))     # 'tac'
```

`reversed(word)` walks the list backwards (Day 1, section 8) and `"".join(...)` glues the characters with nothing in between. `"".join(word[::-1])` does the same. `sorted` also works directly on a string. It treats the string as a sequence of characters and returns a **list** of them, in order:

```
>>> sorted("listen")
['e', 'i', 'l', 'n', 's', 't']
>>> sorted("silent") == sorted("listen")      # same letters, same counts: anagrams
True
>>> "".join(sorted("listen"))                 # glue back if you need a string
'eilnst'
```

8.7 The first character, safely: `s[:1]` vs `s[0]`

`s[0]` raises `IndexError` on an empty string. The slice `s[:1]` gives the first character, or `""` if there is none, and never fails (slices never raise, Day 1). So "capitalise the first letter and lowercase the rest" can be written safely as:

```
w[:1].upper() + w[1:].lower()      # "hELLO" -> "Hello", "" -> ""
```

Similarly, `items[:-1]` is "everything except the last item" and `items[-1]` is "the last item": useful when the last element must be treated differently, like the last line of a justified paragraph, or the last word, which gets no gap after it.

8.8 Checking every position: how many places can a substring start?

To find every (possibly overlapping) place where `sub` occurs in `s`, look at each possible starting position `i` and compare the slice `s[i:i + len(sub)]` with `sub`. The question is which `i`s to try. The last possible start is the one where `sub` ends exactly at the end of `s`: `i = len(s) - len(sub)`. Since `range` stops *before* its end, the loop is:

```
for i in range(len(s) - len(sub) + 1):
    if s[i:i + len(sub)] == sub:
        ...
```

Check it on `s = "aaaa"`, `sub = "aa"`: `range(4 - 2 + 1)` is 0, 1, 2, and the slices are `s[0:2]`, `s[1:3]`, `s[2:4]`, all `"aa"`. Three matches. Forgetting the `+ 1` misses a match at the very end, the usual off-by-one. Combined with the counting pattern from Day 1 (`sum(1 for ... if ...)`), the whole count fits on one line.

8.9 Shrinking until it fits

Sometimes the simplest approach is to start with a candidate that's too big and cut it down until it works. For the longest common prefix of several words, start with the whole first word as the candidate and drop its last character (`prefix = prefix[:-1]`) while another word doesn't start with it:

```
words: "flower", "flow", "flight"
prefix "flower" -> "flow" starts with it? no -> "flowe" -> no -> "flow" -> yes
then "flight": "flow" -> no -> "flo" -> no -> "fl" -> yes
answer: "fl"
```

The loop always ends, because the empty string `""` is a prefix of every string (`"abc".startswith("")` is `True`).

8.10 Sharing out spaces evenly with `divmod`

Full justification has to spread `spaces` extra spaces over `gaps` gaps as evenly as possible, with the leftmost gaps getting one more when it doesn't divide evenly. That's exactly what `divmod` answers (Day 1, section 8):

```
base, extra = divmod(spaces, gaps)
```

`base` is how many spaces *every* gap gets, and `extra` is how many are left over. Those leftovers go one each to the first `extra` gaps. Example: 7 spaces over 3 gaps.

```
divmod(7, 3) = (2, 1)  ->  every gap gets 2, and 1 is left over
gap 0: 2 + 1 = 3       (i < extra, so it gets one more)
gap 1: 2
gap 2: 2               total 3 + 2 + 2 = 7
```

In code, "one more if `i < extra`" is a conditional expression: `base + (1 if i < extra else 0)`. The parentheses are needed. Without them Python would read it as `(base + 1) if i < extra else 0`.

8.11 Default values that are tuples

A function can have a parameter with a default (Day 1, section 5), and that default can be a tuple:

```python
def title_case(s, minor_words=("a", "an", "the", "of")):
    ...
    if w in minor_words: ...
```

`in` works on tuples just like on lists. Why a tuple and not a list? Because a tuple can't be changed, it's a safe default value. Day 4 explains the bug you get with a list as a default.

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 2` after every change; peek at `solutions/` only after 15 minutes stuck.

File	Type	What
`exercises/ex01_basics.py`	implement	palindrome, Caesar cipher, title case, overlapping count, ...
`exercises/ex02_rle.py`	implement	run-length encode/decode (multi-digit counts)
`exercises/ex03_text_format.py`	implement	greedy word-wrap and full justification
`exercises/ex04_fix_me.py`	**debug**	7 string functions, one bug each

Some background for each:

- **ex01** practises the methods from section 2 and the `ord`/`chr` arithmetic from section 3. Several of the functions are small versions of things you'll see in bigger programs, like parsing `"a=1;b=2"` into key/value pairs.
- **ex02** is **run-length encoding (RLE)**, one of the simplest compression methods: replace each run of repeated characters with its length and the character. `"aaabcc"` becomes `"3a1b2c"`. Encoding is pattern 1 from section 6. Decoding is pattern 2, because counts can have several digits: `"12a"` means twelve a's, not "1" then "2a".
- **ex03** is **word wrapping**: split a text into lines of at most `width` characters, putting as many words on each line as fit. "Greedy" means you keep adding words to the current line until the next one doesn't fit, then start a new line. **Justification** then adds extra spaces between the words so every line is exactly `width` characters long, as in a newspaper column. Both are good practice in careful counting: does the space between two words count toward the width? (It does.)
- **ex04** is the exam skill applied to strings: seven small functions, each with exactly one planted bug, most of them from the traps in this document.

Which tools each function needs (section numbers in brackets; “D1” means Day 1):

Function	Tools
`is_palindrome`	`isalnum`, `lower` (2), a comprehension (D1), `[::-1]`
`caesar`	`ord`/`chr` and `%` (3), `"a" <= c <= "z"` (8), building with a list + `join` (5)
`title_case`	`split(" ")`/`join` (2), `enumerate` (8), `w[:1].upper()` (8), `in` on a tuple (8)
`count_overlapping`	the “every starting position” loop (8), `sum(1 for ...)` (D1)
`reverse_each_word`	`isspace` (2), list of characters + `"".join(reversed(...))` (8)
`longest_common_prefix`	`startswith` (2), the “shrink until it fits” loop (8)
`parse_key_values`	`split(";")`, `strip` (2), `partition` + `_` (8), list of tuples (8), `continue` (D1)
`rle_encode`	pattern 1, the index-pointer loop (6), f-strings (4)
`rle_decode`	pattern 2, collecting digits (6), `isdigit` (2), `"a" * n` (8), `raise ValueError` (D1)
`wrap`	`split()` (2), counting lengths: `len(current) + 1 + len(word)`
`justify_line`	`ljust` (2), `divmod` for spreading spaces (8), `enumerate`, `words[:-1]` (8)
`justify`	`wrap`, `justify_line`, `lines[:-1]` and `lines[-1]` (8)

```
python run.py 2
```

End-of-day checklist

Tick every box

- [ ] I know the difference between `split()` and `split(" ")`.
- [ ] I know `strip("abc")` strips a **set of characters**, and `removeprefix` exists.
- [ ] I can write a `while i < len(s)` scanning loop without an infinite loop.
- [ ] I use `repr()` / `f"{x!r}"` when debugging strings.

Exam lens

- Expect edge cases: empty strings, repeated separators, mixed case, and multi-digit run lengths.
- `split()` versus `split(" ")`, `strip` as a character set, and off-by-one slices are planted often.
- When the spec says bytes, compare `len(s)` with `len(s.encode())`; they are not the same.

Tonight’s bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.